

Defining the process and cpu scheduler classes

```
In [1]: class Process:
def __init__(self, pid, arrival_time, burst_time):
    self.pid = pid # Process ID
    self.arrival_time = arrival_time # Time at which the process arrives in the system
    self.burst_time = burst_time # Time required by the process to complete its execution
    self.start_time = None # Time at which the process starts its execution
    self.end_time = None # Time at which the process finishes its execution

def get_waiting_time(self):
    return self.start_time - self.arrival_time

def get_turnaround_time(self):
    return self.end_time - self.arrival_time

class CPU_Scheduler:
def __init__(self):
    self.queue = [] # Queue to store the processes waiting to be executed
    self.current_time = 0 # Current time in the system (in time units)

def add_process(self, process):
    self.queue.append(process) # Add a new process to the end of the queue

def run(self):
    self.queue.sort(key=lambda x: x.arrival_time) # Sort the queue by arrival time
    for process in self.queue:
        print(f"Processing PID {process.pid}...") # Print the process ID
        process.start_time = max(self.current_time, process.arrival_time) # Update the process start time
        self.current_time = process.start_time + process.burst_time # Update the current time by adding the process's burst time
        process.end_time = self.current_time # Update the process end time

def get_avg_waiting_time(self):
    total_waiting_time = sum([process.get_waiting_time() for process in self.queue])
    return total_waiting_time / len(self.queue) # Return the average waiting time for all processes in the queue

def get_avg_turnaround_time(self):
    total_turnaround_time = sum([process.get_turnaround_time() for process in self.queue])
    return total_turnaround_time / len(self.queue) # Return the average turnaround time for all processes in the queue
```

Applying the algorithm

Process	Burst time	Arrival time
P1	6	2
P2	2	5
P3	8	1
P4	3	0
P5	4	4

```
In [2]: # PROCESS(id, arrival, burst)
p1 = Process(1, 2, 6)
p2 = Process(2, 5, 2)
p3 = Process(3, 1, 8)
p4 = Process(4, 0, 3)
p5 = Process(5, 4, 4)

scheduler = CPU_Scheduler()

scheduler.add_process(p1)
scheduler.add_process(p2)
scheduler.add_process(p3)
scheduler.add_process(p4)
scheduler.add_process(p5)

scheduler.run()

print(f"Average waiting time: {scheduler.get_avg_waiting_time()}")
```

Processing PID 4...
Processing PID 3...
Processing PID 1...
Processing PID 5...
Processing PID 2...
Average waiting time: 8.0